

Обектно ориентирано програмиране – Основни принципи

Ивайло Андреев + Claude Sonnet 4.6

17 май 2026

Съдържание

1	Основни принципи на ООП	3
2	Абстракция със структури от данни. Класове и обекти. Декларация на клас и декларация на обект. Основни видове конструктори. Управление на динамичната памет и ресурсите (RAII). Методи	3
2.1	Абстракция със структури от данни	3
2.2	Класове и обекти. Декларация на клас и декларация на обект	4
2.3	Основни видове конструктори	5
2.3.1	Конструктор по подразбиране (Default Constructor)	5
2.3.2	Параметризиран конструктор (Parameterized Constructor)	5
2.3.3	Инициализиращ списък (Member Initializer List)	6
2.3.4	Копиращ конструктор (Copy Constructor)	6
2.3.5	Преместващ конструктор (Move Constructor)	7
2.3.6	Конвертиращ конструктор (Converting Constructor)	7
2.3.7	Деструктор	7
2.4	Управление на динамичната памет и ресурсите (RAII)	8
2.4.1	Дълбоко копиране срещу плитко копиране	8
2.4.2	Правила за управление на специалните методи	8
2.5	Методи – декларация, предаване на параметри, връщане на резултат	9
3	Наследяване. Производни и вложени класове. Достъп до наследените компоненти. Капсулация и скриване на информацията. Статични полета и методи.	9
3.1	Наследяване	9
3.2	Производни класове	10
3.3	Вложени класове	10
3.4	Достъп до наследените компоненти	10
3.4.1	Наследяване (“is-a”) срещу композиция и агрегация (“has-a”)	11
3.5	Капсулация и скриване на информацията	11

3.6	Статични полета и методи	12
-----	------------------------------------	----

1 Основни принципи на ООП

Обектно-ориентираното програмиране (ООП) е програмна парадигма, при която системата се моделира като набор от взаимодействащи си обекти. За разлика от процедурния подход, тук обектите са основните носители на логика и данни.

Четири основни принципа на ООП са:

- **Абстракция** – моделиране на реални обекти чрез представяне само на съществените им характеристики.
- **Капсулация** – скриване на вътрешното състояние; достъпът се осъществява само чрез публичен интерфейс.
- **Наследяване** – нови класове се изграждат върху съществуващи, наследявайки техните характеристики.
- **Полиморфизъм** – унифициран интерфейс за обекти от различни типове; едно действие се реализира различно в зависимост от обекта.

2 Абстракция със структури от данни. Класове и обекти. Декларация на клас и декларация на обект. Основни видове конструктори. Управление на динамичната памет и ресурсите (RAII). Методи

2.1 Абстракция със структури от данни

Абстракцията в ООП е процес на **моделиране на реални обекти** (или концепции) чрез класове. При моделирането ние избираме само съществените за задачата характеристики и игнорираме останалите. Всеки обект се характеризира с три аспекта:

- **Структура / Състояние (член-данни):** Какво *знае* обектът – атрибутите, чрез които се описва. Например за клиент на банка: `name`, `balance`, `accountId`.
- **Поведение (методи):** Какво *прави* обектът – операциите, с които взаимодейства с околния свят. Например: `deposit()`, `withdraw()`, `getBalance()`.
- **Идентичност:** Какво го *отличава* от другите обекти от същия клас. Всяка инстанция е уникална (с различен адрес в паметта), дори ако има еднакви стойности на полетата.

Класът е шаблонът (типът), а обектът е конкретна инстанция. Например класът `BankAccount` моделира банкова сметка, а `aliceAccount` е конкретна сметка на Алис:

```

1 class BankAccount {
2 public:
3     BankAccount(double initial_balance) {
4         balance = (initial_balance >= 0) ? initial_balance : 0;
5     }
6
7     void deposit(double amount) {           // behavior
8         if (amount > 0) balance += amount;
9     }
10
11     bool withdraw(double amount) {          // behavior
12         if (amount > 0 && balance >= amount) {
13             balance -= amount;
14             return true;
15         }
16         return false;
17     }
18
19     double getBalance() const { return balance; } // behavior
20
21 private:
22     double balance; // state
23 };
24
25 BankAccount aliceAccount(500.0); // object (instance)
26 BankAccount bobAccount(200.0);   // different identity, same class

```

Потребителят на класа работи само с публичния интерфейс – той не знае (и не трябва да знае) как точно се съхранява балансът.

2.2 Класове и обекти. Декларация на клас и декларация на обект

Декларацията на клас дефинира нов тип – структурата и поведението му. Сама по себе си не заделя памет.

```

1 // Class declaration
2 class Person {
3 private:
4     std::string name;
5     int age;
6
7 public:
8     // ... methods
9 };

```

Декларацията на обект създава конкретна инстанция от класа и заделя памет за нейните член-данни. Всеки обект има собствено състояние, но споделя дефиницията на методите с останалите.

```

1 // Declaration (and definition) of objects
2 Person p1;
3 Person p2;

```

Обхват на видимост (Scope): Класовете могат да бъдат дефинирани в:

- **Глобален обхват** (на ниво namespace): достъпни в цялата програма след декларацията им.
- **Локален обхват**: клас, дефиниран в тялото на функция, е достъпен само в нея (методите му трябва да бъдат дефинирани inline).

2.3 Основни видове конструктори

Конструкторът е специална член-функция, извиквана автоматично при създаване на обект. Инициализира член-данните и установява валидно начално състояние (класов инвариант). Носи същото име като класа и **няма тип на връщана стойност**.

2.3.1 Конструктор по подразбиране (Default Constructor)

Извиква се без аргументи. Компиляторът го генерира автоматично с празно тяло, **но само ако не сме дефинирали нито един друг конструктор**. Задължителен за създаване на масиви от обекти.

- Сигнатура: `Widget();`

2.3.2 Параметризиран конструктор (Parameterized Constructor)

Приема аргументи за инициализация на член-данните с конкретни стойности.

- Сигнатура: `Widget(int size, const std::string& name);`

Засенчване на имена (Shadowing): Когато параметър съвпада по име с член-данна, предпочитаният подход е инициализиращият списък.

```

1 class A {
2     private:
3         int size;
4     public:
5         // Preferred: member initializer list
6         A(int size) : size(size) {}
7 };
8
9 class B {
10     private:
11         int size;
12     public:
13         // Alternative: this pointer (less efficient)
14         B(int size) {
15             this->size = size;
16         }
17 };

```

Класът може да има **множество конструктори** с различен брой или типове аргументи (претоварване на конструктори):

```

1 class Point {
2 public:
3     Point() : x(0), y(0) {}           // default
4     Point(double x, double y) : x(x), y(y) {} // parameterized
5     Point(double val) : x(val), y(val) {}     // from single value
6 private:
7     double x, y;
8 };

```

2.3.3 Инициализиращ списък (Member Initializer List)

Синтактична конструкция, изпълняваща се **преди** тялото на конструктора. Извършва директна инициализация – за разлика от присвояването в тялото. Може да се ползва за всички член-данни, а не само когато е задължителен.

```

1 class MyClass {
2     int x;
3     const int y;
4 public:
5     // Using member initializer list (preferred):
6     MyClass(int val_x, int val_y)
7         : x(val_x), // initializes x directly
8           y(val_y)  // the only way for const members
9     {
10        // body can be empty or do additional work
11    }
12
13    // Without initializer list (body assignment):
14    MyClass(int val_x_only) {
15        x = val_x_only; // assignment after default-construction of x
16        // y = 0;       // ERROR: cannot assign to const after
17        construction
18    }
19 };

```

Задължителен е в следните случаи:

- Извикване на конструктор на базов клас – единственият начин да се подадат аргументи към родителски конструктор.
- За `const` член-данни – трябва да се инициализират при създаване.
- За членове референции (&).
- За член-данни от класове без конструктор по подразбиране.

2.3.4 Копиращ конструктор (Copy Constructor)

Създава нов обект като копие на съществуващ. Ключов при управление на ресурси (дълбоко копиране). Извиква се при инициализация от друг обект и при подаване/връщане по стойност.

- Сигнатура: `Widget(const Widget& other);`

Разлика между копиращия конструктор и `operator=`: И двата извършват копиране, но се извикват в различен контекст – разграничителят е дали обектът *вече съществува*:

```
1 Widget a;
2 Widget b(a);    // copy constructor: b is a NEW object, initialized from a
3 Widget c = a;    // copy constructor: despite '=', c is NEW (copy-
                    // initialization)
4
5 Widget d;
6 d = a;          // operator=: d ALREADY EXISTS; a's state is assigned into
                    // d
```

Символът `=` **не означава винаги присвояване** – при деклариране на нов обект той извиква копиращия конструктор.

2.3.5 Преместващ конструктор (Move Constructor)

“Открадва” ресурсите от временен изтичащ обект (rvalue), вместо да ги копира скъпо. След преместването оригиналът остава в “празно”, но валидно състояние.

- Сигнатура: `Widget(Widget&& other) noexcept;`
- `noexcept` е критично важен – уверява стандартната библиотека, че операцията е безопасна.

2.3.6 Конвертиращ конструктор (Converting Constructor)

Конструктор с един аргумент, дефиниращ правило за имплицитно преобразуване от друг тип. Маркира се с `explicit`, за да се забрани автоматичното преобразуване.

```
1 class foo {
2 public:
3     foo(int) {} // converting constructor from int to foo
4 };
5
6 void g(foo obj) { }
7
8 // g(5);          // OK: compiler implicitly creates foo(5)
9 // g(foo(5));    // OK: explicit call (required with explicit keyword)
```

2.3.7 Деструктор

Извиква се автоматично при унищожаване на обект. Предназначен да освободи всички ресурси (динамична памет, файлове, мрежови връзки).

- Сигнатура: `~Widget();`

2.4 Управление на динамичната памет и ресурсите (RAII)

RAII (Resource Acquisition Is Initialization) е централен идиом в C++: придобиването на ресурс се обвързва с инициализацията на обект, а освобождаването – с деструктора му. Гарантира автоматично и надеждно управление.

Разширените механизми са:

- **Оператор за присвояване чрез копиране:** `Widget& operator=(const Widget& other);`
- **Оператор за присвояване чрез преместване:** `Widget& operator=(Widget&& other) noexcept;`

2.4.1 Дълбоко копиране срещу плитко копиране

По подразбиране копиращите операции извършват **плитко копиране** (shallow copy) – копират само стойностите бит по бит. При указател се копира адресът, не данните. Това води до:

- Два обекта сочат към един и същ ресурс – неочаквани странични ефекти.
- *Double-free error* – деструкторът освобождава ресурса два пъти.

Решението е ръчно дефинирано **дълбоко копиране** (deep copy). Важно е да се познава разликата между инициализация и присвояване:

```
1 Test t1; // default constructor
2
3 // Forms that invoke the COPY CONSTRUCTOR:
4 Test t2(t1); // direct initialization -- most explicit form
5 Test t3 = t1; // copy-initialization -- despite '=', creates a new object
6
7 // Form that invokes the ASSIGNMENT OPERATOR:
8 t2 = t1; // t2 already exists; t1's state is assigned to t2
```

2.4.2 Правила за управление на специалните методи

На практика класовете следват едно от следните правила:

- **Нищо не дефинираме:** Ако класът не управлява ресурси директно – разчитаме изцяло на компилатора.
- **Голяма четворка:** Ако имаме ръчно управление на ресурси (напр. динамична памет), дефинираме всичките четири: {конструктор по подразбиране, копиращ конструктор, копиращ `operator=`, деструктор}. Дефиниран деструктор **спира** автоматичното генериране на преместващите операции от компилатора.
- **Голяма шестица (C++11+):** Ако добавяме и поддръжка за преместване – дефинираме всичките шест: {конструктор по подразбиране, копиращ конструктор, копиращ `op=`, преместващ конструктор, преместващ `op=`, деструктор}.

2.5 Методи – декларация, предаване на параметри, връщане на резултат

Методите (член-функциите) дефинират поведението на обекта. Декларират се в тялото на класа.

Указателят this: Всеки нестатичен метод получава скрит параметър `this` – указател към обекта, за който е извикан. Чрез него методът достъпва член-данните на конкретната инстанция. Компиляторът трансформира член-функциите до обикновени функции с допълнителен първи параметър `this`:

```
1 struct Point {
2     bool isInFirstQuadrant();    // non-const method
3     bool isOrigin() const;      // const method
4 };
5
6 // Compiler transforms them roughly to:
7 //   bool isInFirstQuadrant(Point* const this)    // non-const
8 //   bool isOrigin(const Point* const this)       // const: cannot
9 //       modify *this
10 //   ~~~~~
11 // pt.isInFirstQuadrant() --> isInFirstQuadrant(&pt)
12 // pt.isOrigin()          --> isOrigin(&pt)
```

Константни методи: Деклариран с `const` накрая, методът обещава да не променя състоянието на обекта. В него `this` е указател към `const` обект. Могат да се извикват върху `const` обекти.

```
1 struct obj {
2     void inspect() const; // Promises not to modify *this
3     void mutate();       // May modify *this
4 };
5
6 void Test(obj& changeable, const obj& unchangeable) {
7     changeable.inspect();    // OK: does not modify a mutable object
8     changeable.mutate();    // OK: modifies a mutable object
9     unchangeable.inspect();  // OK: does not modify an immutable object
10    unchangeable.mutate();    // ERROR: attempt to modify a const object
11 }
```

Изключение: член-данни декларирани с `mutable` могат да бъдат модифицирани дори от `const` методи (например за кеш или брояч на извиквания).

3 Наследяване. Производни и вложени класове. Достъп до наследените компоненти. Капсулация и скриване на информацията. Статични полета и методи.

3.1 Наследяване

Наследяването е механизъм, чрез който нов клас (наследник/производен) се изгражда върху вече съществуващ (базов/родителски). Производният клас автоматично получава всички публични и защитени членове на базовия, може да добавя

нови и да презаписва (override) виртуални методи. Постига се **повторна употреба на код** и организация в йерархия.

3.2 Производни класове

```
1 class Animal {
2 public:
3     void eat() { std::cout << "Eating...\n"; }
4 };
5
6 class Dog : public Animal {    // Dog is a derived class of Animal
7 public:
8     void bark() { std::cout << "Barking...\n"; }
9 };
10 // Dog inherits eat() from Animal and adds its own method bark()
```

3.3 Вложени класове

Вложен клас е клас, дефиниран вътре в друг (външен) клас. Служи за логическа групировка и капсулиране. Вложеният клас има достъп до публичните и защитените **статични** членове на външния; за нестатичните се нуждае от обект на външния клас.

```
1 class Outer {
2 public:
3     class Inner {
4     public:
5         void display() { std::cout << "Inner class\n"; }
6     };
7 };
8
9 Outer::Inner obj;
10 obj.display();
```

3.4 Достъп до наследените компоненти

Достъпът до наследените членове зависи от **вида на наследяването**:

```
1 class Derived : public Base { ... }; // public inheritance
2 class Derived : protected Base { ... }; // protected inheritance
3 class Derived : private Base { ... }; // private inheritance
```

- При **public**: **public** и **protected** членове запазват достъпността си.
- При **protected**: **public** и **protected** членове стават **protected**.
- При **private**: всичко става **private**.

По подразбиране: При `class Derived : Base` е `private`; при `struct Derived : Base` е `public`.

Важно: Производният клас **не може** директно да достъпва `private` членове на базовия – само чрез публични/защитени методи. Наследените методи могат да бъдат презаписвани (`override`), ако са виртуални.

3.4.1 Наследяване (“is-a”) срещу композиция и агрегация (“has-a”)

- **Композиция:** “Силна” зависимост – обект В не може да съществува без А. Реализира се чрез вграден член (В се създава и унищожава заедно с А).
- **Агрегация:** “По-слаба” зависимост – В може да съществува независимо. Реализира се чрез указатели или референции.

```
1 class Car {
2     Engine engine; // composition: Engine is created and destroyed with
3     Car
4 };
5 class Car2 {
6     Engine* engine; // aggregation: Car2 only uses Engine, does not own
7     it
8 public:
9     Car2(Engine* e) : engine(e) {}
10 };
```

Наследяването (“is-a”) е подходящо за специализации при полиморфизъм. Композицията/агрегацията (“has-a”) водят до по-гъвкав дизайн.

3.5 Капсулация и скриване на информацията

Капсулацията е механизмът за обединяване на данни и методи в единна структура (клас), като вътрешното състояние се скрива от външния свят. Достъпът се контролира чрез **модификатори на достъп**:

- `private`: Само от методите на същия клас. По подразбиране в `class`.
- `protected`: От класа и неговите наследници.
- `public`: От всяка точка на програмата. По подразбиране в `struct`.

Практическо приложение са **аксесори** (getters) и **мутатори** (setters):

```
1 class Person {
2     private:
3         int age;
4     public:
5         int getAge() const { return age; } // Accessor (Getter)
6         void setAge(int a) { if (a > 0) age = a; } // Mutator (Setter) with
7         validation
8 };
```

3.6 Статични полета и методи

Статичните членове са асоциирани със самия **клас**, а не с конкретен обект – съществува само едно копие, споделено между всички инстанции.

- **Статични данни:** Декларират се с `static` в класа, но задължително се **дефинират и инициализират извън него**. Съществуват през цялото изпълнение на програмата.
- **Статични методи:** Няма скрит `this`. Достъпват директно само статични членове. Извикват се чрез `ClassName::method()`.

```
1 class Widget {
2 public:
3     Widget() { count++; }
4     ~Widget() { count--; }
5
6     static int getCount() { return count; }
7
8 private:
9     static int count; // declaration
10 };
11
12 int Widget::count = 0; // definition and initialization
13
14 int main() {
15     std::cout << Widget::getCount() << std::endl; // 0
16
17     Widget w1, w2;
18     std::cout << Widget::getCount() << std::endl; // 2
19
20     {
21         Widget w3;
22         std::cout << w1.getCount() << std::endl; // 3 (also accessible
23         // through an object)
24     } // w3 is destroyed here
25
26     std::cout << Widget::getCount() << std::endl; // 2
27 }
```

Статични членове и наследяване: Статичните членове **не се дублират** в класа-наследник:

- **Статични данни:** Едно копие, споделено от базовия клас и всички наследници.
- **Статични методи:** Могат да се извикват и през наследника. **Не могат да бъдат виртуални**. Наследник, дефинирал статичен метод със същото име, го *скрива* (hiding), а не го предефинира (overriding).

static извън контекста на класове: Ключовата дума има две допълнителни употреби:

- **Модификатор на функция / глобална променлива на ниво файл:**
Ограничава видимостта им само до текущата компилационна единица. Двата записа са еквивалентни:

```
1 static void f() { }    // visible only in this translation unit
2
3 namespace {           // equivalent: anonymous namespace
4     void f() { }
5 }
```

- **Модификатор на локална променлива вътре във функция:** Инициализира се **само при първото** влизане и живее на същото място в паметта като глобалните променливи (статична памет), а не на стека.